Milestone 1 Coding Exercises

Explained, solved and extended

Pandas Data Pipelines and FastAPI Applications: the four problems from your Student Practice Preparation Pack, with line-by-line reasoning, alternatives from your syllabus, traps, and more practice with answer keys.

How to use this document

- **Part A and B (Pandas):** Exercises 1 and 2. Same six-function pattern, different nouns. Solutions were run on your sample CSV data and pass test checks.
- **Part C and D (FastAPI):** Exercises 3 and 4. Same API, different nouns. Part D shows exactly what to rename.
- **Part E:** extra practice questions in the same pattern, with answer keys (the Pandas and NumPy ones were executed).
- **Part F:** viva-style questions and a last-night checklist.
- Each function or file has four boxes: **WHY THIS WAY** (blue), **ALTERNATIVES** (green, only things taught in Days 1 to 11), **KEY THINGS TO REMEMBER** (amber) and **TRAPS** (red).

HONEST NOTE ON TESTING

The Pandas solutions (Exercises 1 and 2 and the extra Pandas/NumPy answers) were executed and checked. FastAPI could not be installed in my working environment, so the FastAPI code (Exercises 3 and 4 and the sketches in Part E) is carefully written to the spec but **not executed by me**. Run `pytest -q` on your machine first. If anything fails, send me the error and I will fix it.

The one pattern behind every Pandas question

Step	Question to ask yourself	Pandas tool
Load	Which columns exactly, in which order?	read_csv(usecols=...) then [cols]
Clean	What counts as bad data? Which duplicate wins?	dropna(subset), drop_duplicates(keep)
Merge	Do I keep or drop rows with no match?	merge(how='inner')
Derive	Which new column, rounded when?	df.assign / vectorised arithmetic + round
Aggregate	One row per what? Which statistics?	groupby + named agg
Finalize	Exact column order, rounding, sort, index?	select columns, round, sort_values, reset_index(drop=True)

SQL view of the same pipeline (Day 6 link)

Knowing the SQL twin of each Pandas step helps you explain your choices in a viva.

Pandas	SQL equivalent
df.dropna(subset=['email'])	WHERE email IS NOT NULL
df.drop_duplicates('id', keep='first')	keep first row per id (ROW_NUMBER() OVER (PARTITION BY id ORDER BY input order) = 1)
merge(..., how='inner')	INNER JOIN ... ON a.course_id = b.course_id
merge(..., how='left')	LEFT JOIN
groupby('x').agg(mean, count)	GROUP BY x with AVG(...), COUNT(...)
sort_values('x')	ORDER BY x
df[df.count_col >= 2] after groupby	HAVING COUNT(*) >= 2
df[df.fee > 100] before groupby	WHERE fee > 100

Part A. Exercise 1: Training Enquiry Fee Summary (Pandas)

A1. The problem in plain words

A training institute has a list of enquiries (who asked about which course, quoted fee, discount). The data is dirty: one person has no email, one enquiry ID appears twice, and one enquiry points to a course that does not exist in the catalogue. You must produce a clean report: for each course, the **average fee after discount** and the **number of valid enquiries**.

A2. Trace every row first (this is what you do before writing code)

enquiry_id	course_id	Problem?	Fate
3001	11	none	Kept. Net = 30000 x 0.90 = 27000
3002 (first)	12	none	Kept. Net = 24000 x 0.95 = 22800
3003	13	none	Kept. Net = 20000
3004	14	email missing	**Removed in clean step**
3005	15	none	Kept. Net = 22000 x 0.95 = 20900
3006	11	none	Kept. Net = 32000 x 0.85 = 27200
3007	14	none	Kept. Net = 19000
3002 (second)	12	duplicate ID	**Removed** (keep first, so 24000 row wins, not 25000)
3008	99	course 99 not in catalogue	**Removed by inner join**

Expected result: Cloud Fundamentals 20000.0 (1), Data Analytics 27100.0 (2), Generative AI Basics 20900.0 (1), Python Full Stack 22800.0 (1), Software Testing 19000.0 (1). Data Analytics = (27000 + 27200) / 2 = 27100. **Always do this hand calculation. It is your real test.**

A3. Function by function

1) load_student_enquiries(file_path)

```
def load_student_enquiries(file_path):
    cols = ["enquiry_id", "course_id", "student_name", "email",
            "phone", "quoted_fee", "discount_pct"]
    return pd.read_csv(file_path, usecols=cols)[cols]
```

WHY THIS WAY

The spec says return *exactly* these seven columns and exclude enquiry_date. **usecols** reads only what you ask for, so the unwanted column is never loaded.

usecols does not guarantee the order you list. Pandas returns columns in the file's order. The trailing **[cols]** forces your order. If the CSV already has that order you will not see a difference, but a hidden test file might have shuffled columns.

The list is inside the function (not global) so nothing leaks between functions.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

Read all, then drop: `df = pd.read_csv(p); df = df.drop(columns=['enquiry_date'])`. Fine, but if the file has an extra unexpected column it survives. Selecting with [cols] is safer than dropping.

Read all, then select: `pd.read_csv(p)[cols]`. Same result, loads more data first.

Day 3 link (file I/O): csv.DictReader also works but returns dicts, not a DataFrame, so it breaks the 'every function returns a DataFrame' rule.

KEY THINGS TO REMEMBER

Return a DataFrame. Never print here. Use a relative path passed in as a parameter (no hard-coded 'data/...' inside the function).

Test: `list(df.columns) == cols`.

2) clean_enquiry_data(df)

```
def clean_enquiry_data(df):
    out = df.copy()
    out = out.dropna(subset=["email"])
    out = out.drop_duplicates(subset=["enquiry_id"], keep="first")
    return out.reset_index(drop=True)
```

WHY THIS WAY

df.copy(): the constraints say do not modify the input DataFrame in place. A test may check that the original still has the missing-email row.

subset=['email']: plain `dropna()` deletes a row if *any* column is empty (for example a missing phone). The spec only says missing email.

keep='first' is the default, but writing it makes the intent obvious and protects you if you misremember the default. The spec explicitly says first, not last.

Order of the two steps follows the spec: remove missing emails, then dedupe. This matters. Imagine ID 3002 appears twice and the *first* row has no email. Dropping missing emails first leaves the second row as the only (and therefore first valid) one. Deduping first would keep the bad row and then delete it, losing the ID completely.

reset_index(drop=True): after deleting rows the index has gaps (0,1,2,4,...). drop=True throws the old index away instead of adding it as a column.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

Boolean mask (Day 5 selecting): `df[df['email'].notna()]` is identical to dropna(subset=...). Both are fine.

duplicated(): `df[~df['enquiry_id'].duplicated(keep='first')]`. The ~ means NOT. Same result.

Method chaining: `df.dropna(subset=['email']).drop_duplicates('enquiry_id', keep='first').reset_index(drop=True)` returns a new frame at each step, so it does not modify the input and needs no explicit copy().

Blank emails: if the data contains '' instead of empty cells, convert first: `out['email'] = out['email'].replace('', pd.NA)`, then dropna. Also `.str.strip()` for emails with spaces.

TRAPS

`df.dropna(inplace=True)` modifies the caller's data. Avoid inplace everywhere in this course.

`keep='last'` would keep the 25000 row and give Python Full Stack 25000 instead of 22800.

Forgetting reset_index: a test comparing `list(df.index)` fails.

3) merge_with_courses(enquiries_df, ref_file_path)

```
def merge_with_courses(enquiries_df, ref_file_path):
    courses = pd.read_csv(ref_file_path)
    keep = ["course_id", "course_name", "category",
            "duration_months", "standard_fee"]
    merged = enquiries_df.merge(courses[keep], on="course_id", how="inner")
    return merged.reset_index(drop=True)
```

WHY THIS WAY

how='inner' keeps only rows whose course_id exists on both sides. Enquiry 3008 (course 99) vanishes, exactly as the spec requires.

courses[keep]: the spec lists which reference columns to keep. Selecting them before merging also prevents duplicate-name columns (Pandas would create _x and _y suffixes if both frames had, say, a 'category' column).

The reference file is read *inside* the function from the path parameter, as the signature demands.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

pd.merge(left, right, on='course_id', how='inner'): identical, function style.

join (Day 5): `enq.join(courses.set_index('course_id'), on='course_id', how='inner')`. Works, but more steps to get right.

Filter then map: `df[df.course_id.isin(courses.course_id)]` followed by map for names. Longer, easy to mis-order columns. Merge is the intended tool.

Left join + dropna on course_name: same final rows as inner, but it is the roundabout way. A left join is right when the spec says 'keep unmatched rows'.

KEY THINGS TO REMEMBER

Inner = intersection. Left = keep all left rows, unmatched get NaN.

Key columns must have the same dtype (int vs string will not match). If you see zero rows after a merge, print both dtypes.

Be able to explain: *why does inner join remove the record with an unknown reference ID?* Because there is no matching row on the right side to join to.

4) add_net_quoted_fee(df)

```
def add_net_quoted_fee(df):
    out = df.copy()
    out["net_quoted_fee"] = (
        out["quoted_fee"] * (1 - out["discount_pct"] / 100)
    ).round(2)
    return out
```

WHY THIS WAY

Vectorised: arithmetic on whole columns happens at once. The constraints forbid row-by-row loops.

Parentheses then .round(2): round the finished value, not the pieces. Rounding discount_pct/100 first would give wrong cents.

1 - discount/100: a 10% discount means you pay 90%.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

assign (Day 5): `df.assign(net_quoted_fee=(df.quoted_fee * (1 - df.discount_pct/100)).round(2))` returns a new DataFrame, so no copy() is needed.

NumPy (Day 4): `np.round(df['quoted_fee'] * (1 - df['discount_pct']/100), 2)`. Same numbers, shows the NumPy/Pandas link.

Subtraction form: `quoted_fee - quoted_fee*discount_pct/100`. Same result.

apply(axis=1) with a lambda: gives the same numbers but runs a Python function per row. It is slow and **violates the 'no row-by-row' rule**. Do not use it here.

KEY THINGS TO REMEMBER

Test value to remember: quoted_fee 1000 and discount 33 gives 670.0.

Dividing by 100 in Python 3 gives a float, so integer inputs become float outputs. That is why the final averages print as 20000.0.

5) aggregate_by_course(df)

```
def aggregate_by_course(df):
    return (
        df.groupby("course_name", as_index=False)
        .agg(avg_net_quoted_fee=("net_quoted_fee", "mean"),
             enquiry_count=("enquiry_id", "count"))
        .reset_index(drop=True)
    )
```

WHY THIS WAY

Named aggregation `new_name=(column, function)` creates the exact output column names in the same step. No renaming needed.

as_index=False: keeps course_name as a normal column. Without it, course_name becomes the index and the column disappears.

count on enquiry_id counts valid rows per course. IDs are never missing after cleaning, so this equals the number of rows.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

agg with a dict then rename: `df.groupby('course_name')['net_quoted_fee'].agg(['mean','count']).reset_index().rename(columns={'mean':'avg_net_quoted_fee','count':'enquiry_count'})`. Works; named aggregation is cleaner.

size() instead of count(): `groupby(...).size()` counts rows including NaN; count() counts non-missing values in that column. Here equal.

Two separate groupbys then merge: works but is longer and error-prone.

SQL (Day 6): `SELECT course_name, AVG(net_quoted_fee), COUNT(*) FROM ... GROUP BY course_name`.

TRAPS

Do not take the mean of already-rounded averages from another step. Average the net fees, then round once at the end.

groupby silently drops rows whose group key is NaN (default dropna=True). Not a problem here after the inner join, but important in Part E.

6) finalize_summary(df)

```
def finalize_summary(df):
    out = df[["course_name", "avg_net_quoted_fee", "enquiry_count"]].copy()
    out["avg_net_quoted_fee"] = out["avg_net_quoted_fee"].round(2)
    return out.sort_values("course_name").reset_index(drop=True)
```

WHY THIS WAY

Selecting the three columns in a list *forces* the exact order the spec demands.

Round the average to two decimals. Averages like 27100.0 are fine; averages like 22833.3333 become 22833.33.

sort_values('course_name') sorts A to Z by default. After sorting the index is shuffled, so reset_index(drop=True) comes *last*.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

round via dict: `df.round({'avg_net_quoted_fee': 2})`.

sort by index trick: not needed; sort_values is clearer.

KEY THINGS TO REMEMBER

Order of the last two operations: sort first, reset index second.

Column order, rounding, sorting, clean index. Memorise these four words for the final function of every pipeline.

7) The main block

Under `if __name__ == '__main__':` call the six functions in order, passing each result to the next, and print only the last. Importing main.py in tests must not run the pipeline, which is exactly what this guard does. File paths live in this block, not inside the functions.

A4. Complete solution (tested)

```
import pandas as pd


def load_student_enquiries(file_path):
    cols = ["enquiry_id", "course_id", "student_name", "email",
            "phone", "quoted_fee", "discount_pct"]
    return pd.read_csv(file_path, usecols=cols)[cols]


def clean_enquiry_data(df):
    out = df.copy()
    out = out.dropna(subset=["email"])
    out = out.drop_duplicates(subset=["enquiry_id"], keep="first")
    return out.reset_index(drop=True)


def merge_with_courses(enquiries_df, ref_file_path):
    courses = pd.read_csv(ref_file_path)
    keep = ["course_id", "course_name", "category",
            "duration_months", "standard_fee"]
    merged = enquiries_df.merge(courses[keep], on="course_id", how="inner")
    return merged.reset_index(drop=True)


def add_net_quoted_fee(df):
    out = df.copy()
    out["net_quoted_fee"] = (
        out["quoted_fee"] * (1 - out["discount_pct"] / 100)
    ).round(2)
    return out


def aggregate_by_course(df):
    return (
        df.groupby("course_name", as_index=False)
        .agg(avg_net_quoted_fee=("net_quoted_fee", "mean"),
             enquiry_count=("enquiry_id", "count"))
        .reset_index(drop=True)
    )
```

```
def finalize_summary(df):
    out = df[["course_name", "avg_net_quoted_fee", "enquiry_count"]].copy()
    out["avg_net_quoted_fee"] = out["avg_net_quoted_fee"].round(2)
    return out.sort_values("course_name").reset_index(drop=True)


if __name__ == "__main__":
    df = load_student_enquiries("data/training_enquiries.csv")
    df = clean_enquiry_data(df)
    df = merge_with_courses(df, "data/training_courses.csv")
    df = add_net_quoted_fee(df)
    df = aggregate_by_course(df)
    df = finalize_summary(df)
    print(df)
```

A5. Rules in the constraints, and why they exist

Rule	Reason
No row-by-row loops	Vectorised operations are faster and are what Day 5 teaches. Graders may scan for for/iterrows/apply(axis=1).
Do not modify inputs in place	Functions must be independent and re-runnable; tests compare inputs before and after.
No global intermediate DataFrames	Hidden state makes tests order-dependent.
No hard-coded names or counts	Hidden tests use different CSV data. 'Python Full Stack' typed in your code fails the moment the data changes.
Exact function and parameter names	Tests call main.add_net_quoted_fee(...) and keyword arguments by name.
Importing main.py must not run the pipeline	pytest imports your file. Without the __main__ guard it would run and print.

A6. Mistakes that cost marks, with the effect

Mistake	What the grader sees
dropna() without subset	Rows with a missing phone also vanish, so counts are too low
keep='last'	Wrong fee for the duplicated ID, so wrong average
how='left' or 'outer'	Course 99 survives with NaN course_name; counts and groupby behave oddly
Rounding the discount fraction	Cents are wrong in some rows
No as_index=False	course_name is missing as a column, so KeyError in finalize
Averaging after rounding twice	Tiny rounding differences fail equality tests
Leaving enquiry_date in	Column list does not match
Printing inside a function	Violates 'do not print from pipeline functions'
No __main__ guard	Import runs the pipeline

Part B. Exercise 2: Customer Service Request Summary (Pandas)

B1. Same pattern, new nouns

Exercise 1	Exercise 2
load_student_enquiries	load_service_requests
clean_enquiry_data (enquiry_id)	clean_service_requests (request_id)
merge_with_courses (course_id)	merge_with_services (service_id)
course_name, category, duration_months, standard_fee	service_name, category, duration_days, standard_fee
aggregate_by_course, enquiry_count	aggregate_by_service, request_count
finalize: course_name, avg_net_quoted_fee, enquiry_count	finalize: service_name, avg_net_quoted_fee, request_count

The logic is identical, so reading the spec carefully for **names** is the whole job. Two new details: the reference column is **duration_days** (not months) and the customer_name column holds service-like text in the sample file. **Do not group by customer_name**; the spec groups by **service_name**, which comes from the catalogue after the merge.

B2. Trace the sample rows

request_id	service_id	Problem?	Fate
4101	21	none	Kept: 1200 x 0.90 = 1080
4102	22	**email missing**	Removed. So Laptop Repair has only 4107 left
4103 (first)	23	none	Kept: 1800
4104	24	none	Kept: 900 x 0.90 = 810
4105	21	none	Kept: 1400 x 0.85 = 1190
4106	25	none	Kept: 3200 x 0.95 = 3040
4107	22	none	Kept: 2300
4103 (second)	23	duplicate ID	Removed (keep first, so 1800 not 1900)
4108	88	service 88 not in catalogue	Removed by inner join

B3. Complete solution (tested) and its output

```
import pandas as pd


def load_service_requests(file_path):
    cols = ["request_id", "service_id", "customer_name", "email",
            "phone", "quoted_fee", "discount_pct"]
    return pd.read_csv(file_path, usecols=cols)[cols]


def clean_service_requests(df):
    out = df.dropna(subset=["email"]).drop_duplicates(subset=["request_id"], keep="first")
    return out.reset_index(drop=True)


def merge_with_services(requests_df, ref_file_path):
    cat = pd.read_csv(ref_file_path)
    keep = ["service_id", "service_name", "category", "duration_days", "standard_fee"]
    return requests_df.merge(cat[keep], on="service_id", how="inner").reset_index(drop=True)


def add_net_quoted_fee(df):
    return df.assign(net_quoted_fee=(df["quoted_fee"] * (1 - df["discount_pct"] / 100)).round(2))


def aggregate_by_service(df):
    return (df.groupby("service_name", as_index=False)
              .agg(avg_net_quoted_fee=("net_quoted_fee", "mean"),
                   request_count=("request_id", "count")))


def finalize_summary(df):
    out = df[["service_name", "avg_net_quoted_fee", "request_count"]].copy()
    out["avg_net_quoted_fee"] = out["avg_net_quoted_fee"].round(2)
    return out.sort_values("service_name").reset_index(drop=True)


if __name__ == "__main__":
    df = load_service_requests("data/service_requests.csv")
    df = clean_service_requests(df)
    df = merge_with_services(df, "data/service_catalogue.csv")
```

```
    df = add_net_quoted_fee(df)
    df = aggregate_by_service(df)
    df = finalize_summary(df)
    print(df)
```

service_name	avg_net_quoted_fee	request_count
Cybersecurity Check	3040.0	1
Data Backup	1800.0	1
Home Wi-Fi Setup	1135.0	2
Laptop Repair	2300.0	1
Printer Setup	810.0	1

Hand check: Home Wi-Fi Setup = (1080 + 1190) / 2 = 1135. Laptop Repair is 2300 only, because request 4102 had no email.

B4. What changed in the code, and the style alternatives I used

- **clean_service_requests** uses method chaining (`df.dropna(...).drop_duplicates(...)`). Each method returns a new DataFrame, so the input is never modified and no copy() is needed. Either style is acceptable.
- **add_net_quoted_fee** uses `df.assign(...)`. Same idea: returns a new frame. Day 5 syllabus covers both column creation styles.
- **merge_with_services** selects catalogue columns first, same reason as Exercise 1 (avoid _x and _y suffixes, control the columns kept).
- Both styles were verified to give identical results (I compared dropna vs boolean mask, drop_duplicates vs ~duplicated, named agg vs dict agg).

KEY THINGS TO REMEMBER

Write the exercise-specific names from the spec into a checklist before coding. Copy and paste them; do not retype.

If the exam gives you a third variant (orders, tickets, employees) you can answer it using the table in Part A section A3 without new thinking.

Part C. Exercise 3: Consultant and Practice Area API (FastAPI)

C1. The problem in plain words

A consulting firm has **practice areas** (for example Data Engineering on floor 3). Each practice area has many **consultants**. You build a small web API so an administrator can: create a practice area, add a consultant under it, fetch a practice area together with its consultants (with optional filter and sort), and update a consultant. This is a classic **parent-child (one-to-many)** design: one parent, many children, each child belongs to exactly one parent.

practice_areas (parent)	consultants (child)
id (PK)	id (PK)
name	name, designation
floor_number (>= 1)	consultation_fee (>= 300), experience_years (>= 0)
	practicearea_id (FK to practice_areas.id)

C2. What happens to one request (learn this flow, it explains every status code)

Step	Who does it	Failure result
1. Route matching (method + path)	FastAPI	404 / 405 for unknown path or method
2. Convert path and query parameters (parent_id must be int)	FastAPI + type hints	422
3. Validate the JSON body against the Pydantic schema	Pydantic	**422** (this happens *before* your function runs)
4. Open a DB session	Depends(get_db)	-
5. Your logic: look up parent/child, apply filter and sort	You	raise HTTPException(404)
6. db.add, db.commit, db.refresh	SQLAlchemy	-
7. Convert the return value with response_model	Pydantic	-
8. Close the DB session (finally block)	get_db	-

KEY THINGS TO REMEMBER

422 comes before 404. If you test 'create a child under a missing parent', send a *valid* body, otherwise you get 422 and not the 404 you wanted to test.

You write no code for 422. FastAPI and Pydantic produce it. You only write code for 404.

C3. File by file

database.py: the connection

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///./consulting.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

WHY THIS WAY

create_engine is the connection to the SQLite file consulting.db. The spec names the database.

check_same_thread=False: SQLite normally refuses to be used from a different thread than the one that created the connection; FastAPI may use several threads.

sessionmaker builds sessions. **autocommit=False** means nothing is saved until you call commit(); **autoflush=False** avoids surprise partial writes.

Base is the parent class all ORM models inherit from; it keeps the list of tables.

get_db is a *generator dependency*: it opens a session, **yield**s it to the endpoint, and the **finally** block closes it after the response, even if an exception occurred. One session per request.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

Context manager: `with SessionLocal() as db:` inside a helper. Equivalent, but FastAPI's Depends pattern with yield is what Day 9 and Day 11 taught.

PostgreSQL (Day 9): only DATABASE_URL changes (`postgresql://user:pass@host/db`) and check_same_thread is removed. This is why the engine is isolated in one file.

One global session: never do this; concurrent requests would share transactions.

TRAPS

Forgetting the finally/close leaks connections. Creating the session inside each endpoint by hand repeats code; use the dependency.

models.py: the tables

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from database import Base


class PracticeArea(Base):
    __tablename__ = "practice_areas"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String, nullable=False)
    floor_number = Column(Integer, nullable=False)
    consultants = relationship("Consultant", back_populates="practice_area",
                               cascade="all, delete-orphan")


class Consultant(Base):
    __tablename__ = "consultants"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String, nullable=False)
    designation = Column(String, nullable=False)
    consultation_fee = Column(Integer, nullable=False)
    experience_years = Column(Integer, nullable=False)
    practicearea_id = Column(Integer, ForeignKey("practice_areas.id"), nullable=False)
    practice_area = relationship("PracticeArea", back_populates="consultants")
```

WHY THIS WAY

Each class is a table (__**tablename**__ must match the spec: practice_areas and consultants).

ForeignKey('practice_areas.id') is the database-level link (string is *table.column*).

relationship + back_populates is the Python-level link in both directions: `area.consultants` gives the list, `consultant.practice_area` gives the parent. The spec requires a **bidirectional** relationship.

nullable=False on practicearea_id: a consultant must belong to a parent.

cascade='all, delete-orphan': if a practice area is ever deleted, its consultants go with it. Not tested here but good design; be ready to explain it.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

backref: `relationship('Consultant', backref='practice_area')` creates the reverse side automatically with one line. Shorter, but the spec asks for explicit bidirectional, so use back_populates on both.

SQLModel (Day 9 syllabus mentions SQLAlchemy / SQLModel): combines the ORM model and Pydantic schema into one class. Valid, but this exercise explicitly wants separate models.py and schemas.py.

Alembic (Day 9): instead of `create_all`, schema changes are applied as versioned migrations. create_all is enough for this exam.

TRAPS

Names must match the spec: the FK column is practicearea_id (no underscore between 'practice' and 'area'). A different spelling breaks tests that check the response or database.

back_populates names must mirror each other exactly or SQLAlchemy raises a mapper error.

schemas.py: validation and response shapes

```
from typing import List
from pydantic import BaseModel, Field, ConfigDict


class PracticeAreaCreate(BaseModel):
    name: str = Field(min_length=1)
    floor_number: int = Field(ge=1)


class ConsultantCreate(BaseModel):
    name: str = Field(min_length=1)
    designation: str = Field(min_length=1)
    consultation_fee: int = Field(ge=300)
    experience_years: int = Field(ge=0)


class ConsultantOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    designation: str
    consultation_fee: int
    experience_years: int


class PracticeAreaOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    floor_number: int


class PracticeAreaWithConsultants(PracticeAreaOut):
    consultants: List[ConsultantOut] = []
```

WHY THIS WAY

Schemas are the API's *contract*. ORM models describe the database; Pydantic schemas describe what is accepted and returned. Keeping them separate means you never accidentally leak internal columns.

Create schemas (what the client sends) hold the validation rules: `Field(min_length=1)` rejects empty strings; `ge=300` means fee must be at least 300; `ge=1` and `ge=0` likewise. Breaking a rule makes FastAPI answer 422 automatically.

One ConsultantCreate for both create and update (the spec says so). A PUT replaces all four editable fields.

Out schemas (what the server returns) include the generated **id**. They do not include practicearea_id (the spec's output lists omit it).

from_attributes=True lets Pydantic read values from an ORM object (attributes) instead of requiring a dict.

PracticeAreaWithConsultants inherits PracticeAreaOut and adds the list of children; this gives the nested JSON the GET endpoint needs.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

Pydantic v1 style: `class Config: orm_mode = True` instead of model_config. Check your installed version with `pip show pydantic`. Do not mix both styles.

Constrained types: `fee: conint(ge=300)`, `name: constr(min_length=1)`. Same effect, older style.

Annotated style (v2): `NonEmpty = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1)]`. This also rejects a name made only of spaces.

field_validator: write a custom rule, for example reject whitespace-only names.

TRAPS

`Field(min_length=1)` accepts the string `' '` (spaces). If the exam says 'non-empty' and tests whitespace, use the strip_whitespace version.

Using Optional fields in the Create schema would make a missing field valid; the spec says missing fields give 422.

main.py: the four endpoints

```
from typing import Optional
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session

import models, schemas
from database import engine, get_db

models.Base.metadata.create_all(bind=engine)
app = FastAPI()

SORT_FIELDS = {"name": models.Consultant.name,
               "consultation_fee": models.Consultant.consultation_fee}


@app.post("/practice-areas", response_model=schemas.PracticeAreaOut, status_code=201)
def create_practice_area(payload: schemas.PracticeAreaCreate, db: Session = Depends(get_db)):
    obj = models.PracticeArea(**payload.model_dump())
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj


@app.post("/practice-areas/{parent_id}/consultants",
          response_model=schemas.ConsultantOut, status_code=201)
def create_consultant(parent_id: int, payload: schemas.ConsultantCreate,
                      db: Session = Depends(get_db)):
    parent = db.get(models.PracticeArea, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Practice area not found")
    child = models.Consultant(**payload.model_dump(), practicearea_id=parent_id)
    db.add(child)
    db.commit()
    db.refresh(child)
    return child


@app.get("/practice-areas/{parent_id}", response_model=schemas.PracticeAreaWithConsultants)
def get_practice_area(parent_id: int, designation: Optional[str] = None,
                      sort: Optional[str] = None, db: Session = Depends(get_db)):
```

```
    parent = db.get(models.PracticeArea, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Practice area not found")
    if sort is not None and sort.strip() != "" and sort not in SORT_FIELDS:
        raise HTTPException(status_code=404, detail="Invalid sort field")
    q = db.query(models.Consultant).filter(models.Consultant.practicearea_id == parent_id)
    if designation is not None and designation.strip() != "":
        q = q.filter(models.Consultant.designation == designation)
    if sort:
        q = q.order_by(SORT_FIELDS[sort])
    else:
        q = q.order_by(models.Consultant.id)
    return {"id": parent.id, "name": parent.name,
            "floor_number": parent.floor_number, "consultants": q.all()}


@app.put("/practice-areas/{parent_id}/consultants/{child_id}",
         response_model=schemas.ConsultantOut)
def update_consultant(parent_id: int, child_id: int, payload: schemas.ConsultantCreate,
                      db: Session = Depends(get_db)):
    parent = db.get(models.PracticeArea, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Practice area not found")
    child = db.get(models.Consultant, child_id)
    if child is None or child.practicearea_id != parent_id:
        raise HTTPException(status_code=404, detail="Consultant not found")
    for k, v in payload.model_dump().items():
        setattr(child, k, v)
    db.commit()
    db.refresh(child)
    return child


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

Why each endpoint is written this way

POST /practice-areas

- `status_code=201` in the decorator: the default is 200, and the spec demands 201 for creation.
- `payload.model_dump()` turns the validated Pydantic object into a dict, so `PracticeArea(**dict)` builds the ORM object. (Pydantic v1: `.dict()`.)
- **add, commit, refresh**: add stages the row, commit saves it, refresh reloads it so the database-generated **id** is available in the response. Skip refresh and id can be missing.

POST /practice-areas/{parent_id}/consultants

- First check the parent exists: `db.get(models.PracticeArea, parent_id)` looks up by primary key and returns None if missing. Then `raise HTTPException(404, ...)`.
- The child's **practicearea_id comes from the URL**, not the body (the body has no such field). That is how ownership is enforced.
- Use **raise**, not **return**, for errors. Returning a dict with an error message sends status 200 or 201.

GET /practice-areas/{parent_id} (the interesting one)

- **designation**: `Optional[str] = None` makes it an optional query parameter (`?designation=Senior`). If missing or blank (`.strip() == ''`) there is no filter; otherwise exact match with == (case-sensitive).
- **sort**: allowed values are the two keys of the SORT_FIELDS dictionary. Anything else raises 404 with a short message (this odd 404 is what the spec demands). The check happens *before* querying so a bad sort never silently falls back to another order.
- The dictionary maps the public name to the ORM column. This is an **allow-list**: user input is never used to build code or column names directly, which is safer than `getattr(models.Consultant, sort)`.
- The query is built in steps: start with children of this parent, add the optional filter, add ordering, run with `.all()`. Filtering in SQL is better than loading everything into Python.
- Default order (no sort given) is by id so results are stable.
- The function returns a **dict** with the filtered children instead of the ORM parent. If you returned the parent object, `parent.consultants` would always contain *all* children and ignore the filter.

PUT /practice-areas/{parent_id}/consultants/{child_id}

- Three separate failure cases, all 404: parent missing; child missing; child exists but **belongs to a different parent** (`child.practicearea_id != parent_id`). The third check is the one students forget; without it you could edit another parent's consultant by using the wrong URL.
- It updates the four editable fields in a loop with `setattr` and never touches practicearea_id, so a child cannot be moved to another parent (spec).
- **404 not 403**: a child of another parent is treated as 'not found under this parent', which is what the spec says.

The __main__ block: uvicorn runs only when you execute `python main.py`. A test importing main must not start a server (the spec repeats this). Host 0.0.0.0 and port 8080 are required.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

db.query(...).filter(...).first() instead of `db.get(...)`. Same result; db.get is shorter for primary keys. (`db.query(Model).get(id)` is the older SQLAlchemy 1.x form.)

Filter and sort in Python using the relationship: `kids = parent.consultants` then a list comprehension for the filter and `sorted(kids, key=lambda c: c.name)`. Uses Day 1 to 3 Python skills and works for small data, but SQL ordering is preferred and was what the exercise expects ('ORM queries').

Query parameter validation with Query (Day 8): `sort: Optional[Literal['name','consultation_fee']]` would give 422 for invalid values. Do not use it here: the spec demands **404**.

if/elif for sort instead of a dictionary: valid, longer.

JSONResponse(status_code=404, content=...) instead of HTTPException: works, but HTTPException is the standard Day 9 'error signalling' approach.

Custom exception handler (Day 9): to change the shape of all 404 JSON bodies globally.

selectinload (eager loading): loads children in one extra query; useful if you return many parents. Not needed for one parent.

TRAPS

Wrong order of checks: test 'child belongs to other parent' needs the parent to exist and the child to exist first.

Returning 400 or 200 with an error message. The spec lists exact status codes.

Forgetting db.refresh after commit, so a field like id is None.

Typing parent_id as str. A non-integer then does not give the 404 you expect.

C4. The tests, explained

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from database import Base, get_db
from main import app

engine = create_engine("sqlite://", connect_args={"check_same_thread": False},
                       poolclass=StaticPool)
TestingSession = sessionmaker(bind=engine, autoflush=False, autocommit=False)


def override_get_db():
    db = TestingSession()
    try:
        yield db
    finally:
        db.close()


app.dependency_overrides[get_db] = override_get_db
client = TestClient(app)


@pytest.fixture(autouse=True)
def fresh_db():
    Base.metadata.drop_all(bind=engine)
    Base.metadata.create_all(bind=engine)
    yield


def make_parent(name="Data", floor=3):
    return client.post("/practice-areas", json={"name": name, "floor_number": floor}).json()


def kid(name="A", des="Senior", fee=800, exp=5):
    return {"name": name, "designation": des, "consultation_fee": fee, "experience_years": exp}
```

```
def test_create_parent():
    r = client.post("/practice-areas", json={"name": "Data", "floor_number": 3})
    assert r.status_code == 201 and r.json()["id"] == 1


def test_create_child_and_missing_parent():
    p = make_parent()
    r = client.post(f"/practice-areas/{p['id']}/consultants", json=kid())
    assert r.status_code == 201 and r.json()["name"] == "A"
    assert client.post("/practice-areas/999/consultants", json=kid()).status_code == 404


def test_get_nested_filter_sort():
    p = make_parent()
    for n, d, f in [("Zed", "Senior", 900), ("Amy", "Junior", 500), ("Bob", "Senior", 700)]:
        client.post(f"/practice-areas/{p['id']}/consultants", json=kid(n, d, f))
    base = f"/practice-areas/{p['id']}"
    assert len(client.get(base).json()["consultants"]) == 3
    seniors = client.get(base, params={"designation": "Senior"}).json()["consultants"]
    assert {c["name"] for c in seniors} == {"Zed", "Bob"}
    assert len(client.get(base, params={"designation": "  "}).json()["consultants"]) == 3
    names = [c["name"] for c in client.get(base, params={"sort": "name"}).json()["consultants"]]
    assert names == ["Amy", "Bob", "Zed"]
    fees = [c["consultation_fee"] for c in client.get(base, params={"sort": "consultation_fee"}).json()["consultants"]]
    assert fees == sorted(fees)
    assert client.get(base, params={"sort": "bogus"}).status_code == 404
    assert client.get("/practice-areas/999").status_code == 404


def test_update_cases():
    p1, p2 = make_parent("A"), make_parent("B", 2)
    c = client.post(f"/practice-areas/{p1['id']}/consultants", json=kid()).json()
    r = client.put(f"/practice-areas/{p1['id']}/consultants/{c['id']}", json=kid("New", "Lead", 1000, 9))
    assert r.status_code == 200 and r.json()["name"] == "New" and r.json()["consultation_fee"] == 1000
    assert client.put(f"/practice-areas/999/consultants/{c['id']}", json=kid()).status_code == 404
    assert client.put(f"/practice-areas/{p1['id']}/consultants/999", json=kid()).status_code == 404
    assert client.put(f"/practice-areas/{p2['id']}/consultants/{c['id']}", json=kid()).status_code == 404
```

```
@pytest.mark.parametrize("body", [
    {"name": "", "designation": "S", "consultation_fee": 800, "experience_years": 1},
    {"name": "A", "designation": "", "consultation_fee": 800, "experience_years": 1},
    {"name": "A", "designation": "S", "consultation_fee": 299, "experience_years": 1},
    {"name": "A", "designation": "S", "consultation_fee": 800, "experience_years": -1},
    {"name": "A", "designation": "S", "consultation_fee": "abc", "experience_years": 1},
    {"name": "A", "designation": "S", "experience_years": 1},
])
def test_invalid_child_422(body):
    p = make_parent()
    assert client.post(f"/practice-areas/{p['id']}/consultants", json=body).status_code == 422


def test_invalid_parent_422():
    assert client.post("/practice-areas", json={"name": "X", "floor_number": 0}).status_code == 422
    assert client.post("/practice-areas", json={"name": "", "floor_number": 1}).status_code == 422
```

WHY THIS WAY

TestClient calls your app in memory; no server is needed.

Separate in-memory database + dependency_overrides[get_db]: tests must not touch consulting.db. FastAPI lets you replace the get_db dependency during tests (Day 11 dependency injection).

StaticPool makes all connections share one in-memory SQLite database; without it each connection would see a new empty database.

autouse fixture that drops and creates tables gives every test a clean start, so tests do not depend on order (the spec demands this).

parametrize runs one test body over many invalid bodies, covering all 422 cases without copy-paste.

Helper functions make_parent and kid avoid repetition and keep test data out of the endpoints.

ALTERNATIVES (mostly from your Day 1 to 11 syllabus)

One test file section per endpoint instead of grouped tests. Fine, but keep each test independent.

pytest tmp_path with a file database instead of in-memory. Slower; same idea.

httpx.Client(app=...): TestClient wraps httpx; either works.

KEY THINGS TO REMEMBER

Every spec rule needs at least one test: 201, 404 (three update cases and sort), 422 (six body cases), nested GET, filter, sort asc, update success.

Assert on specifics (`r.json()['id'] == 1`, order of names), not just status codes.

C5. Where the Day 10 and Day 11 topics would attach (viva questions)

Topic	How it would extend this API
Middleware and CORS (Day 10)	A middleware that adds an X-Process-Time header; CORSMiddleware to allow a browser front-end on another origin.
Redis caching (Day 10)	Cache the GET /practice-areas/{id} response for a few seconds; invalidate on POST/PUT for that parent.
Auth: JWT, OAuth2, RBAC (Day 10 and 11)	Protect POST/PUT with a dependency that decodes the token; only role 'admin' may create or update ('Administrators' in the spec).
Testing strategies (Day 11)	Unit tests (schemas) vs integration tests (TestClient + DB); fixtures and mocking.
Dependency injection (Day 11)	get_db is one; another could be get_current_user.
Alembic migrations (Day 9)	Add a new column to consultants without dropping the table.

Part D. Exercise 4: Trainer and Skill Group API (FastAPI)

Exercise 4 is Exercise 3 with new names. Nothing about the logic changes. Your only job is a careful, complete rename. This is the exact mapping:

Exercise 3	Exercise 4
practice_areas (table), PracticeArea (class)	skill_groups, SkillGroup
consultants, Consultant	trainers, Trainer
consultation_fee	session_fee
practicearea_id	skillgroup_id
/practice-areas ...	/skill-groups ...
consulting.db	learning.db
PracticeAreaCreate / Out / WithConsultants	SkillGroupCreate / Out / WithTrainers
ConsultantCreate / Out	TrainerCreate / Out
sort values: name, consultation_fee	sort values: name, session_fee
floor_number, designation, experience_years	unchanged

RENAME TRAPS

Everything is renamed **consistently**: table name, class name, relationship names, foreign key column, route paths, error messages, test helper names, JSON field names in tests.

Sort allow-list: the dictionary key must become `session_fee`, otherwise ?sort=session_fee returns 404.

The spec text for Exercise 4 says *skillgroup_id* (no underscore between 'skill' and 'group'), like practicearea_id before.

D1. database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///./learning.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

D2. models.py

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from database import Base


class SkillGroup(Base):
    __tablename__ = "skill_groups"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String, nullable=False)
    floor_number = Column(Integer, nullable=False)
    trainers = relationship("Trainer", back_populates="skill_group",
                            cascade="all, delete-orphan")


class Trainer(Base):
    __tablename__ = "trainers"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String, nullable=False)
    designation = Column(String, nullable=False)
    session_fee = Column(Integer, nullable=False)
    experience_years = Column(Integer, nullable=False)
    skillgroup_id = Column(Integer, ForeignKey("skill_groups.id"), nullable=False)
    skill_group = relationship("SkillGroup", back_populates="trainers")
```

D3. schemas.py

```
from typing import List
from pydantic import BaseModel, Field, ConfigDict


class SkillGroupCreate(BaseModel):
    name: str = Field(min_length=1)
    floor_number: int = Field(ge=1)


class TrainerCreate(BaseModel):
    name: str = Field(min_length=1)
    designation: str = Field(min_length=1)
    session_fee: int = Field(ge=300)
    experience_years: int = Field(ge=0)


class TrainerOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    designation: str
    session_fee: int
    experience_years: int


class SkillGroupOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    floor_number: int


class SkillGroupWithTrainers(SkillGroupOut):
    trainers: List[TrainerOut] = []
```

D4. main.py

```
from typing import Optional
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session

import models, schemas
from database import engine, get_db

models.Base.metadata.create_all(bind=engine)
app = FastAPI()

SORT_FIELDS = {"name": models.Trainer.name,
               "session_fee": models.Trainer.session_fee}


@app.post("/skill-groups", response_model=schemas.SkillGroupOut, status_code=201)
def create_skill_group(payload: schemas.SkillGroupCreate, db: Session = Depends(get_db)):
    obj = models.SkillGroup(**payload.model_dump())
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj


@app.post("/skill-groups/{parent_id}/trainers",
          response_model=schemas.TrainerOut, status_code=201)
def create_trainer(parent_id: int, payload: schemas.TrainerCreate,
                   db: Session = Depends(get_db)):
    parent = db.get(models.SkillGroup, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Skill group not found")
    child = models.Trainer(**payload.model_dump(), skillgroup_id=parent_id)
    db.add(child)
    db.commit()
    db.refresh(child)
    return child


@app.get("/skill-groups/{parent_id}", response_model=schemas.SkillGroupWithTrainers)
def get_skill_group(parent_id: int, designation: Optional[str] = None,
                    sort: Optional[str] = None, db: Session = Depends(get_db)):
```

```
    parent = db.get(models.SkillGroup, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Skill group not found")
    if sort is not None and sort.strip() != "" and sort not in SORT_FIELDS:
        raise HTTPException(status_code=404, detail="Invalid sort field")
    q = db.query(models.Trainer).filter(models.Trainer.skillgroup_id == parent_id)
    if designation is not None and designation.strip() != "":
        q = q.filter(models.Trainer.designation == designation)
    if sort:
        q = q.order_by(SORT_FIELDS[sort])
    else:
        q = q.order_by(models.Trainer.id)
    return {"id": parent.id, "name": parent.name,
            "floor_number": parent.floor_number, "trainers": q.all()}


@app.put("/skill-groups/{parent_id}/trainers/{child_id}",
         response_model=schemas.TrainerOut)
def update_trainer(parent_id: int, child_id: int, payload: schemas.TrainerCreate,
                   db: Session = Depends(get_db)):
    parent = db.get(models.SkillGroup, parent_id)
    if parent is None:
        raise HTTPException(status_code=404, detail="Skill group not found")
    child = db.get(models.Trainer, child_id)
    if child is None or child.skillgroup_id != parent_id:
        raise HTTPException(status_code=404, detail="Trainer not found")
    for k, v in payload.model_dump().items():
        setattr(child, k, v)
    db.commit()
    db.refresh(child)
    return child


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

The tests are the Exercise 3 tests with the same replacements (paths `/skill-groups`, field `session_fee`, key `trainers`). **Practice:** do the rename yourself on paper first, then compare with this listing. A good rename check is a text search for any leftover 'consult' or 'practice' after you finish.

Part E. More practice in the same pattern

E1. Pandas pipeline practice (with answer keys)

Practice P1: Orders revenue summary

Files: orders.csv (order_id, customer_id, qty, unit_price, coupon_pct, order_date) and customers.csv (customer_id, customer_name, city). Write: load_orders, clean_orders (drop duplicate order_id keeping first, **drop rows with qty <= 0**), merge_with_customers (inner), add_order_total (`qty * unit_price * (1 - coupon_pct/100)`, 2 decimals), aggregate_by_customer (total_revenue = sum, order_count = count), finalize_summary (sort by **total_revenue descending, ties by customer_name ascending**, clean index).

Answer key (tested on a small sample):

```
def clean_orders(df):
    out = df.drop_duplicates("order_id", keep="first")
    out = out[out["qty"] > 0]
    return out.reset_index(drop=True)

def merge_with_customers(df, ref_file_path):
    ref = pd.read_csv(ref_file_path)[["customer_id", "customer_name", "city"]]
    return df.merge(ref, on="customer_id", how="inner").reset_index(drop=True)

def add_order_total(df):
    return df.assign(order_total=(df["qty"] * df["unit_price"]
                                  * (1 - df["coupon_pct"] / 100)).round(2))

def aggregate_by_customer(df):
    return df.groupby("customer_name", as_index=False).agg(
        total_revenue=("order_total", "sum"), order_count=("order_id", "count"))

def finalize_summary(df):
    return (df.sort_values(["total_revenue", "customer_name"], ascending=[False, True])
              .reset_index(drop=True))
```

KEY THINGS TO REMEMBER

Multi-column sort: `ascending=[False, True]` lines up with the column list.

A boolean mask like `df['qty'] > 0` is the alternative to dropping rows by index. Filtering is **WHERE**; it happens *before* groupby.

Check on sample: customer Asha has orders 2 x 100 at 10% = 180 and 3 x 40 at 20% = 96, total 276.0 and count 2.

Practice P2: Employees by department (left join and the groupby trap)

Files: employees (emp_id, dept_id, salary), departments (dept_id, dept_name). Report: dept_name, avg_salary, headcount, only departments with **at least 2** employees, sorted by avg_salary descending.

```
m = emp.merge(dept, on="dept_id", how="left")        # keeps employee with unknown dept
g = m.groupby("dept_name", as_index=False).agg(
        avg_salary=("salary", "mean"), headcount=("emp_id", "count"))
result = (g[g["headcount"] >= 2]                     # HAVING COUNT(*) >= 2
          .sort_values("avg_salary", ascending=False)
          .reset_index(drop=True))
```

TRAPS

groupby drops NaN keys by default. The employee with dept_id 9 (not in departments) gets dept_name NaN after the left join, and is silently excluded from the groups. If the spec wants an 'Unknown' group, either `fillna({'dept_name': 'Unknown'})` before groupby or pass `dropna=False` to groupby.

Filtering after aggregation (headcount >= 2) is SQL HAVING; filtering before is WHERE. Be able to say which one the question needs.

Practice P3: NumPy + Pandas mix (marks and grades)

students.csv has student_id, name, maths, physics, chemistry. Missing marks are possible and student_id may repeat. Write: **clean_marks** (drop duplicate ids keep first; fill missing marks with that subject's mean), **add_average** (average of the three subjects, 2 decimals), **add_grade** (A if avg >= 75, B if >= 50, else C, using np.select), **zscores** (returns a NumPy array of z-scores of the average) and a count of students with z > 1.

```
import numpy as np, pandas as pd
subjects = ["maths", "physics", "chemistry"]

def clean_marks(df):
    out = df.drop_duplicates("student_id", keep="first").copy()
    out[subjects] = out[subjects].fillna(out[subjects].mean())   # column-wise means
    return out.reset_index(drop=True)

def add_average(df):
    return df.assign(average=df[subjects].mean(axis=1).round(2))

def add_grade(df):
    conds = [df["average"] >= 75, df["average"] >= 50]
    return df.assign(grade=np.select(conds, ["A", "B"], default="C"))

def zscores(df):
    a = df["average"].to_numpy(dtype=float)
    return (a - a.mean()) / a.std()          # NumPy std is population std (ddof=0)
```

KEY THINGS TO REMEMBER

`axis=1` averages across a *row*; `axis=0` works down a column. For Pandas the default of `.mean()` is axis=0, so `df[subjects].mean()` gives one mean per subject, which is what fillna needs.

`np.select(conditions, choices, default)` picks the *first* matching condition, so order the conditions from strictest to loosest.

Pandas `.std()` divides by n-1; NumPy `.std()` divides by n. Say which one you used.

Sample check: averages 70, 41.67, 95 give grades B, C, A and z-scores about 0.05, -1.25, 1.20 (one student above 1).

Practice P4 to P6: do these from the spec only (no answer key; apply the pattern)

- **P4 Hotel bookings:** bookings (booking_id, room_type_id, guest, email, nightly_rate, discount_pct, nights) and room_types (room_type_id, room_type, capacity). Remove missing email; keep **last** duplicate booking_id; add `net_amount = nightly_rate * nights * (1 - discount_pct/100)`; report per room_type: total_net_amount and booking_count, sorted by total descending.
- **P5 Support tickets:** tickets (ticket_id, agent_id, email, minutes_spent, priority) and agents (agent_id, agent_name, team). Drop missing email and duplicates (keep first); left join; report per team: avg_minutes and ticket_count; teams with fewer than 3 tickets removed; unknown agents shown as team 'Unassigned'.
- **P6 Library loans:** loans (loan_id, book_id, member_email, days_loaned, fine_per_day, waiver_pct) and books (book_id, title, genre). Compute `fine = days_loaned * fine_per_day * (1 - waiver_pct/100)`, rounded; per genre: total fine and number of loans; top 3 genres only.

For each: first write the row-by-row trace table like in A2, then code, then compare your hand-calculated totals with the output.

E2. NumPy practice with answers (executed)

Task	Answer
Average mark per student (rows = students)	`marks.mean(axis=1).round(2)`
Average per subject	`marks.mean(axis=0)`
Subject index with the best average	`marks.mean(axis=0).argmax()`
Rows where all marks >= 40	`marks[(marks >= 40).all(axis=1)]`
Grace marks: raise anything below 35 to 35, new array	`np.where(marks < 35, 35, marks)` (or np.maximum(marks, 35))
Min-max scale each column to 0 to 1	`(marks - marks.min(0)) / (marks.max(0) - marks.min(0))` (broadcasting)
Replace NaN with column mean	`np.where(np.isnan(a), np.nanmean(a, axis=0), a)`
Top 3 values and their indices	`idx = np.argsort(a)[-3:][::-1]; a[idx]`
Running total and first day above 1000	`c = sales.cumsum(); (c > 1000).argmax()` (check c.max() > 1000 first)
Count values above the mean	`(a > a.mean()).sum()`
Unique values sorted by frequency	`v, n = np.unique(a, return_counts=True); v[np.argsort(-n)]`

E3. FastAPI practice (build each from a blank folder)

Practice F1: Library API (Shelf and Book) with delete

Parent: shelves (id, label, floor >= 1). Child: books (id, title, author, price >= 50, pages >= 1). Same four endpoints as Exercise 3, plus **DELETE /shelves/{shelf_id}/books/{book_id}** returning 204 with the same ownership checks. GET supports `author` (exact) filter and sort by `title` or `price`.

```
@app.delete("/shelves/{shelf_id}/books/{book_id}", status_code=204)
def delete_book(shelf_id: int, book_id: int, db: Session = Depends(get_db)):
    shelf = db.get(models.Shelf, shelf_id)
    if shelf is None:
        raise HTTPException(status_code=404, detail="Shelf not found")
    book = db.get(models.Book, book_id)
    if book is None or book.shelf_id != shelf_id:
        raise HTTPException(status_code=404, detail="Book not found")
    db.delete(book)
    db.commit()
    # return nothing: 204 means "No Content"
```

KEY THINGS TO REMEMBER

204 responses must have no body. Test with `r.status_code == 204` and then confirm a GET no longer lists the book.

Sketch only; I could not run FastAPI here, so test it yourself.

Practice F2: Team and Member with price range and pagination

Parent: teams. Child: members (name, role, rate >= 300, experience >= 0). GET adds `min_rate` and `max_rate` filters and `skip` / `limit` pagination; invalid limit (below 1 or above 100) must give 422.

```
from fastapi import Query

@app.get("/teams/{team_id}", response_model=schemas.TeamWithMembers)
def get_team(team_id: int,
             role: Optional[str] = None,
             min_rate: Optional[int] = None,
             max_rate: Optional[int] = None,
             skip: int = Query(0, ge=0),
             limit: int = Query(50, ge=1, le=100),
             db: Session = Depends(get_db)):
    ...
    q = db.query(models.Member).filter(models.Member.team_id == team_id)
    if role and role.strip():
        q = q.filter(models.Member.role == role)
    if min_rate is not None:
        q = q.filter(models.Member.rate >= min_rate)
    if max_rate is not None:
        q = q.filter(models.Member.rate <= max_rate)
    members = q.order_by(models.Member.id).offset(skip).limit(limit).all()
```

KEY THINGS TO REMEMBER

Query(ge=1, le=100) is how a numeric *query parameter* gets validation, giving 422 automatically. (Day 8 query parameters.)

Offset/limit must come **after** order_by so pages are stable.

Practice F3: Warehouse and Product with PATCH and case-insensitive filter

Parent: warehouses. Child: products (name, category, unit_price >= 1, stock >= 0). Add **PATCH** (partial update: only the fields sent change) and make the category filter case-insensitive.

```
class ProductPatch(BaseModel):
    name: Optional[str] = Field(default=None, min_length=1)
    category: Optional[str] = Field(default=None, min_length=1)
    unit_price: Optional[int] = Field(default=None, ge=1)
    stock: Optional[int] = Field(default=None, ge=0)

# inside the PATCH endpoint, after the same parent/child/ownership checks:
for k, v in payload.model_dump(exclude_unset=True).items():
    setattr(product, k, v)

# case-insensitive filter:
from sqlalchemy import func
q = q.filter(func.lower(models.Product.category) == category.strip().lower())
```

KEY THINGS TO REMEMBER

exclude_unset=True separates 'field not sent' from 'field sent as null'. Without it you would overwrite untouched fields with None.

PUT replaces all fields (all required). PATCH changes only some (all optional). Know the difference.

Practice F4: change the rules, not the code (read-the-spec drill)

- Exercise 3 but fee minimum 500 and sort accepts `experience_years` as a third option. Which lines change? (Schema `ge`, SORT_FIELDS entry, and tests for 499/500.)
- Exercise 3 but an invalid sort must now return **422**. What changes? (Use `Literal['name','consultation_fee']` in the query parameter type; remove the manual 404.)
- Exercise 3 but a parent with the same name must be rejected with **409**. Where do you add the check and what test proves it?

Part F. Viva questions and last-night checklist

F1. Questions to answer out loud in under 30 seconds each

Question	Short answer
Why inner join and not left join in Exercise 1?	The spec says remove unmatched records. Inner keeps only matching keys.
Why dedupe before aggregating?	Otherwise duplicated rows inflate the count and distort the mean.
Why copy() or method chaining?	To avoid modifying the caller's DataFrame (a stated constraint).
usecols vs drop(columns=...)?	usecols reads only needed columns; drop reads all then removes. Selecting [cols] also fixes the order.
count vs size?	count skips NaN in that column; size counts rows.
Why as_index=False?	Keeps the group key as a normal column, so the output has the required columns.
Why vectorised and not apply?	Faster, simpler, and required by the exercise.
What does reset_index(drop=True) do?	Rebuilds a clean 0..n-1 index and discards the old one.
What is a Pydantic model for?	Validates and shapes data entering and leaving the API.
422 vs 404?	422: the request data is invalid. 404: the requested resource does not exist.
Why Depends(get_db)?	One DB session per request, automatically closed; easy to replace in tests.
commit vs refresh?	commit saves to the database; refresh reloads the row to get generated values such as id.
Why check child.practicearea_id != parent_id?	To stop editing another parent's child through the wrong URL; spec says 404.
Why from_attributes=True?	So Pydantic can serialise SQLAlchemy objects.
Why an in-memory test database?	Tests must be isolated and repeatable and must not change real data.
Why if __name__ == '__main__'?	Run server or pipeline only when executed directly, never on import.

F2. Last-night checklist

- I can write the six Pandas functions from a blank file in 25 minutes and the result matches my hand calculation.
- I can write the four FastAPI files plus tests for a new parent/child pair in 40 minutes.
- I copy names (functions, columns, tables, routes, fields) from the spec, I do not retype them.
- I check the output columns and their order, the index, and rounding on every function.
- I test 201, 200, 404 (all cases), 422 (all cases) and nothing relies on test order.
- No debug prints, no hard-coded data, no in-place edits, __main__ guards in place.
- I run the whole file once from a fresh terminal before submitting.

Tell me which part felt hardest and I will write fresh questions with answer keys for that part.